

Mastering SPI

Serial Peripheral Interface — A Complete Reference Guide
Signal Lines • Clock Modes • Transaction Timing • Interview Q&A

Prepared for automotive embedded software engineering practice
(sensor/EEPROM/flash interfacing, ECU peripheral communication)

Table of Contents

- Part 1 — SPI Fundamentals
- Part 2 — SPI Clock Modes (CPOL / CPHA)
- Part 3 — SPI Transaction / Frame Structure (bit-level)
- Part 4 — Multi-Slave Topologies
- Part 5 — Practical Engineering Notes
- Part 6 — Interview Questions & Answers (50+ questions, categorized)

Part 1 — SPI Fundamentals

1.1 What is SPI and why it's used

SPI (Serial Peripheral Interface) is a synchronous, full-duplex, master-slave serial communication protocol originally introduced by Motorola. Unlike CAN (multi-master, message-based, arbitrated), SPI is strictly **single-master-drives-the-clock**, point-to-point (or point-to-multipoint with separate select lines), and has no built-in addressing, CRC, or arbitration — the master decides who talks by physically selecting one slave at a time.

Why it's everywhere in embedded/automotive designs:

- **Simplicity:** no complex protocol stack — just a shift register clocked in and out.
- **Speed:** SPI commonly runs at several MHz to tens of MHz, far faster than I2C or classic CAN, since there's no arbitration overhead and it's a simple push-pull point-to-point link.
- **Full-duplex:** data flows in both directions simultaneously on the same clock edge, unlike I2C's half-duplex bus.
- **Low silicon cost:** just a shift register and a few pins on the slave side — this is why it's the go-to for cheap peripherals: EEPROMs, flash memory, ADCs/DACs, sensors, display controllers, and external CAN/FD controller chips (e.g., MCP2515-style SPI-to-CAN bridges) commonly found alongside your ABS/TCS ECU hardware.

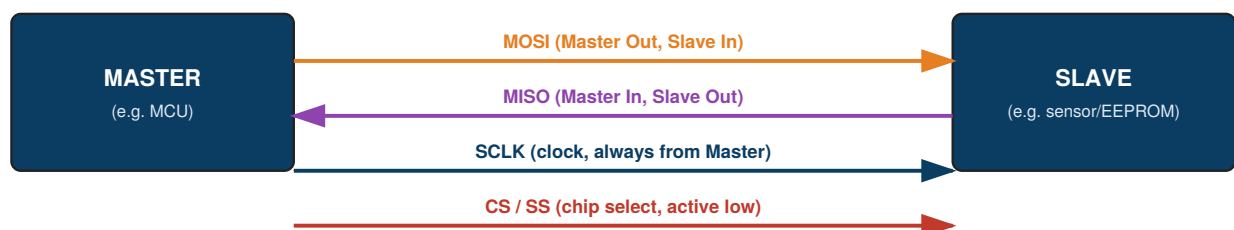
1.2 The Four Signal Lines

Signal	Full name	Driven by	Purpose
MOSI	Master Out, Slave In	Master	Carries data from master to slave
MISO	Master In, Slave Out	Slave	Carries data from slave to master
SCLK (or SCK)	Serial Clock	Master (always)	Clocks each bit in/out; slave has no independent clock — it's entirely clock-slaved to the master
CS / SS	Chip Select / Slave Select	Master	Active-low (typically) line that "wakes up" one specific slave to participate in the transaction; all non-selected slaves must tri-state their MISO output

Some newer device naming conventions (adopted after Motorola's original naming fell out of favor) use **SDO/SDI** (Serial Data Out/In) or **COPI/CIPO** (Controller Out Peripheral In / Controller In Peripheral Out) instead of MOSI/MISO — same electrical function, just renamed for inclusive terminology. Datasheets from 2020 onward increasingly use SDO/SDI or COPI/CIPO; older and many current automotive datasheets still say MOSI/MISO.

1.3 Basic Topology — Single Master, Single Slave

SPI — Single Master, Single Slave Topology



4 wires total. Master always generates SCLK. Data shifts out on MOSI/MISO simultaneously (full duplex).

Four wires connect master to slave directly. Because SCLK is generated solely by the master and there's no clock recovery needed on the slave side (unlike asynchronous UART), SPI slaves can be extremely simple hardware — often just a shift register

with no onboard oscillator at all.

1.4 Full-Duplex Operation — What This Actually Means

On every single SCLK pulse, **one bit shifts out of the master on MOSI AND one bit shifts out of the slave on MISO, at the same time**. This means every SPI transaction is inherently a simultaneous "write and read" — even if you only care about reading from the slave (e.g., reading a sensor register), the master still has to clock out *something* on MOSI (often a dummy byte like 0xFF or 0x00) to generate the clock pulses that pull the response out of the slave.

This trips up a lot of engineers new to SPI: there's no such thing as a "read-only" SPI transaction at the electrical level. Every byte clocked is a byte sent AND a byte received, always. The application/driver just chooses to ignore the received garbage on a "write" and ignore the transmitted dummy on a "read."

Worked example — full-duplex byte exchange

Master wants to read a status register from a slave IC. Typical sequence: master sends command byte `0x05` (a "read status" opcode) while simultaneously receiving a don't-care byte from the slave (say `0xFF`, since the slave hasn't started responding meaningfully yet). Then the master sends a dummy byte `0x00` (just to generate clocks) while the slave now drives its actual status byte, e.g. `0x42`, back on MISO. Two clock-out bytes, two clock-in bytes, only one of each direction is "useful" data — this is completely normal SPI behavior, not a bug.

Part 2 — SPI Clock Modes (CPOL / CPHA)

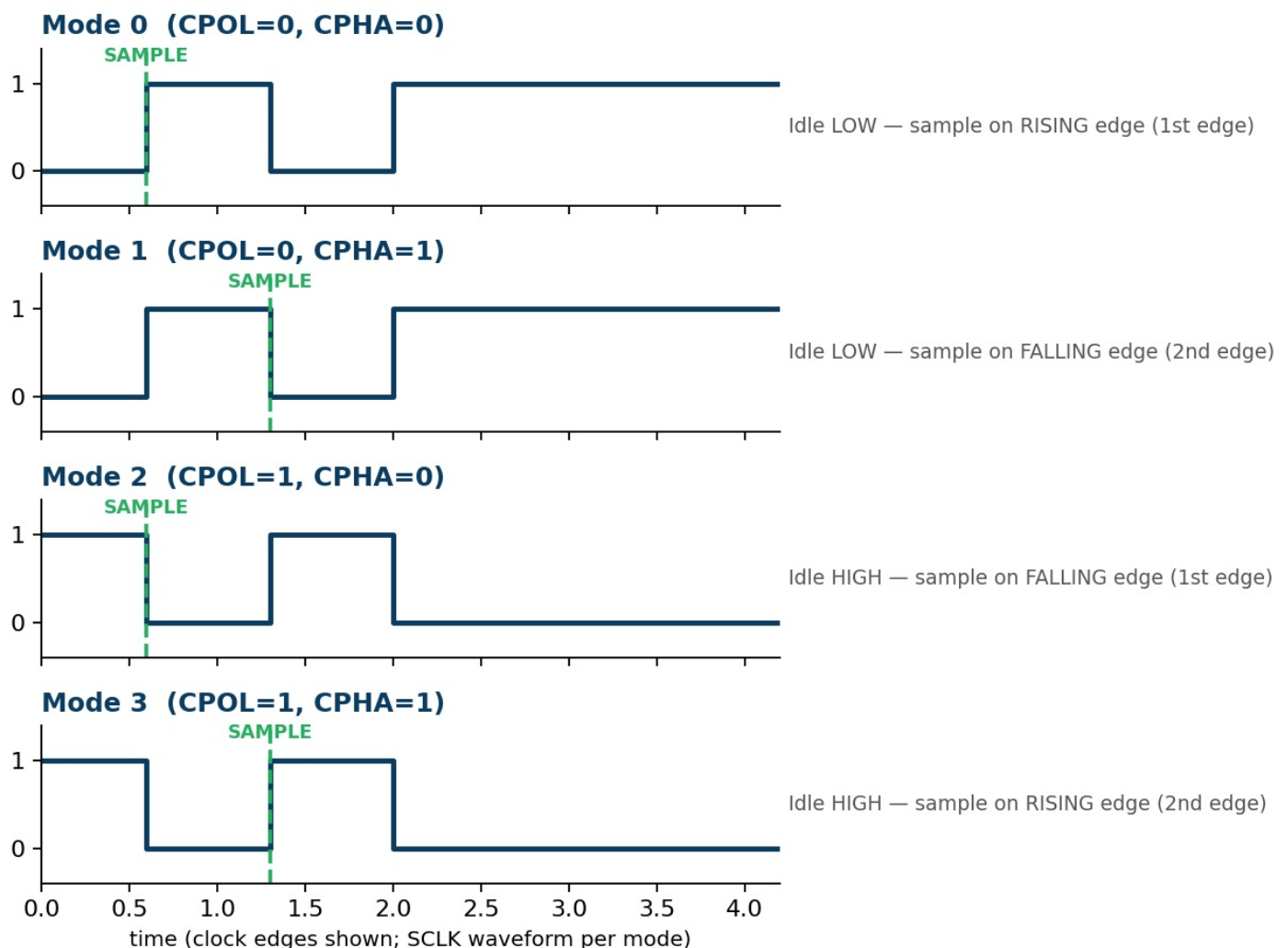
2.1 Why Modes Exist

Different slave chips expect the clock to idle at different levels and expect data to be sampled at different clock edges. Two configuration bits define this, and both master and slave **MUST** agree on the same mode or the bits will be sampled at the wrong instant, silently corrupting every transaction (a very common real-world bring-up bug).

Bit	Name	Meaning
CPOL	Clock Polarity	0 = clock idles LOW between transactions. 1 = clock idles HIGH between transactions.
CPHA	Clock Phase	0 = data is sampled on the FIRST clock edge after CS goes active. 1 = data is sampled on the SECOND clock edge.

2.2 The Four Modes — Timing Comparison

The Four SPI Clock Modes (CPOL / CPHA)



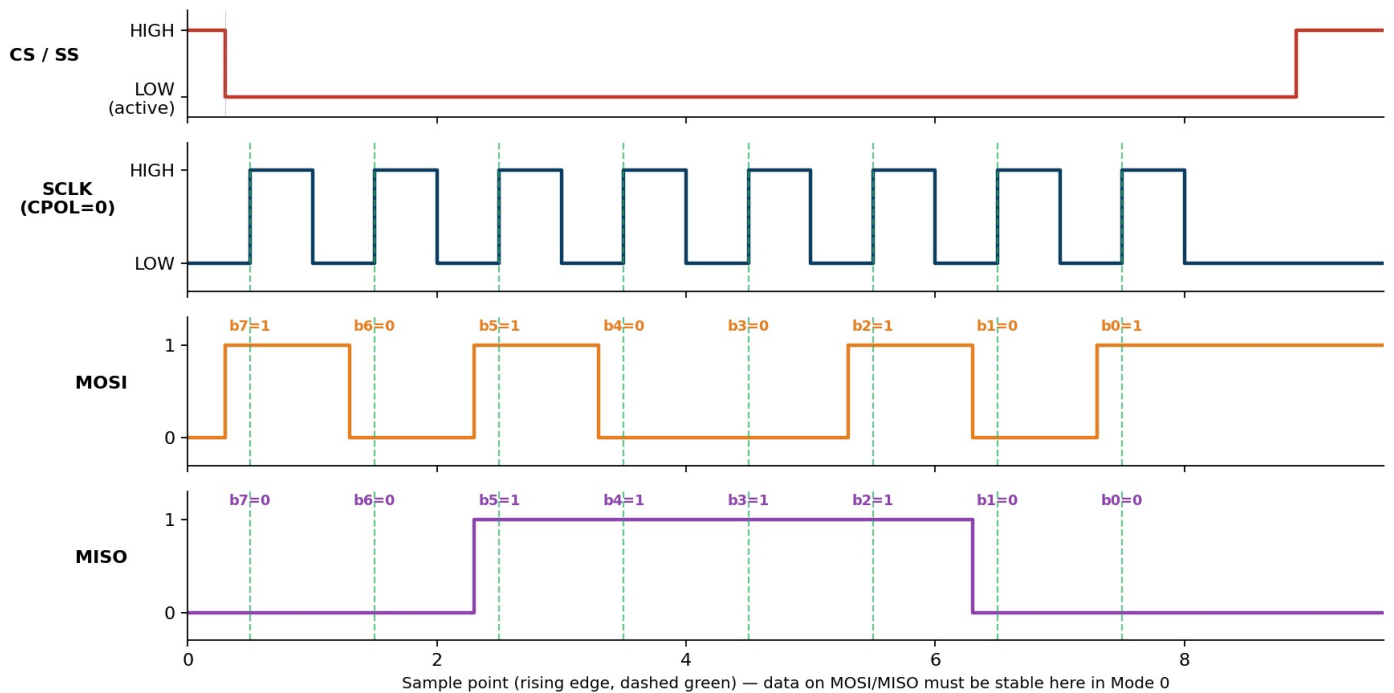
Mode	CPOL	CPHA	Clock idle state	Data sampled on	Common usage
Mode 0	0	0	LOW	Rising edge (1st edge)	Most common default — most EEPROMs, flash chips, many sensors
Mode 1	0	1	LOW	Falling edge (2nd edge)	Less common, some specific ADC/DAC families

Mode 2	1	0	HIGH	Falling edge (1st edge)	Less common
Mode 3	1	1	HIGH	Rising edge (2nd edge)	Common alternative default — some sensor and memory families prefer this over Mode 0

Memorize the pattern rather than the table: **CPOL picks the idle level** (0=low, 1=high). **CPHA picks which of the two edges per bit-cell is the sampling edge** (0=first edge, 1=second edge). Data is always driven (changed) on the edge OPPOSITE to the sampling edge, so it has time to settle before being sampled.

2.3 Worked Example — Single Byte Transfer in Mode 0

SPI Mode 0 (CPOL=0, CPHA=0) — Single Byte Transfer: Master sends 0xA5, Slave responds 0x3C



This shows a complete Mode 0 (CPOL=0, CPHA=0) transaction: the master sends byte 0xA5 (binary 1010 0101) on MOSI while the slave simultaneously sends back 0x3C (binary 0011 1100) on MISO. Reading the diagram:

- **CS** goes LOW first — this is what tells the slave "pay attention, a transaction is starting" and un-tri-states its MISO output.
- **SCLK** idles LOW (CPOL=0) before and after the byte, and produces exactly 8 pulses, one per bit.
- **MOSI** changes state just before each rising edge (on the preceding falling edge) so the new bit value is stable and ready by the time the rising edge (sample point, dashed green) arrives.
- **MISO** behaves identically but in the opposite direction — the slave changes its output on the falling edge, ready to be sampled by the master on the next rising edge.
- Both bytes transfer MSB-first here (the most common convention, though some devices support LSB-first — always check the datasheet).
- **CS** returns HIGH after the 8th bit, ending the transaction and re-tri-stating the slave's MISO.

Interview-important takeaway: master and slave sample the bus at exactly the same instant (the same clock edge) — that's what makes it "synchronous." There's no start/stop bit or baud-rate negotiation like UART; the clock IS the timing reference for everything.

Part 3 — SPI Transaction / Frame Structure

3.1 There Is No Fixed "Frame Format" — By Design

Unlike CAN (which has a rigidly standardized frame with SOF/ID/CRC/etc.) or UART (start/stop/parity bits), SPI has **no protocol-defined frame format at all** at the electrical/data-link level. SPI only defines: how CS behaves, how SCLK behaves (mode), and that data shifts MSB-first or LSB-first. Everything above that — command bytes, address bytes, data length, checksums — is entirely up to the specific slave device's datasheet. This is both SPI's biggest strength (extremely flexible, minimal overhead) and its biggest gotcha (you must read each device's datasheet individually; there's no universal "SPI packet" to learn once).

3.2 Typical Application-Layer Transaction Shapes

Even though SPI itself doesn't mandate structure, most real slave devices follow one of a few common patterns layered on top of raw SPI:

Pattern	Typical byte layout	Example devices
Command + Data	[Opcode byte][Data byte(s)...]	Most SPI EEPROMs/Flash (e.g., opcode 0x03 = Read, 0x02 = Write)
Register Address + R/W bit + Data	[Addr byte with MSB = R/W flag] [Data byte]	Many sensor ICs (accelerometers, gyros) — e.g., address byte's top bit = 1 for read, 0 for write
Command + Address + Dummy + Data	[Opcode][Addr byte(s)][Dummy byte for turnaround][Data...]	SPI NOR Flash memory (e.g., opcode 0x0B "Fast Read" needs a dummy byte for the flash's internal read latency before data comes out)

Worked example — reading a register from a typical sensor IC

Say a sensor's datasheet specifies: to read register 0x0F, send a byte with bit 7 set to 1 (read flag) OR'd with the 7-bit address, followed by a dummy byte to clock out the response.

Clock cycle group	MOSI byte sent	MISO byte received	Meaning
1st byte	0x8F (0x80 0x0F)	0xFF (don't care — slave hasn't decoded the address yet)	Master requests read of register 0x0F
2nd byte	0x00 (dummy, just to generate clocks)	0x37 (actual register value)	Slave has now decoded the address from byte 1 and drives the real register value back

CS stays LOW across both bytes (this is one continuous transaction of 16 clock pulses); it only goes HIGH after the full exchange completes.

3.3 Bit Order: MSB-First vs LSB-First

Most SPI devices shift MSB-first (the most significant bit of each byte goes out first), but some (a minority) use LSB-first. This is a per-device configuration setting on the master's SPI peripheral (most MCU SPI hardware blocks have an LSB-first/MSB-first control bit) and, like CPOL/CPHA, must match what the slave device expects — getting this wrong silently reverses every byte's bit order, producing plausible-looking but completely wrong data.

3.4 Multi-Byte Transfers and CS Behavior

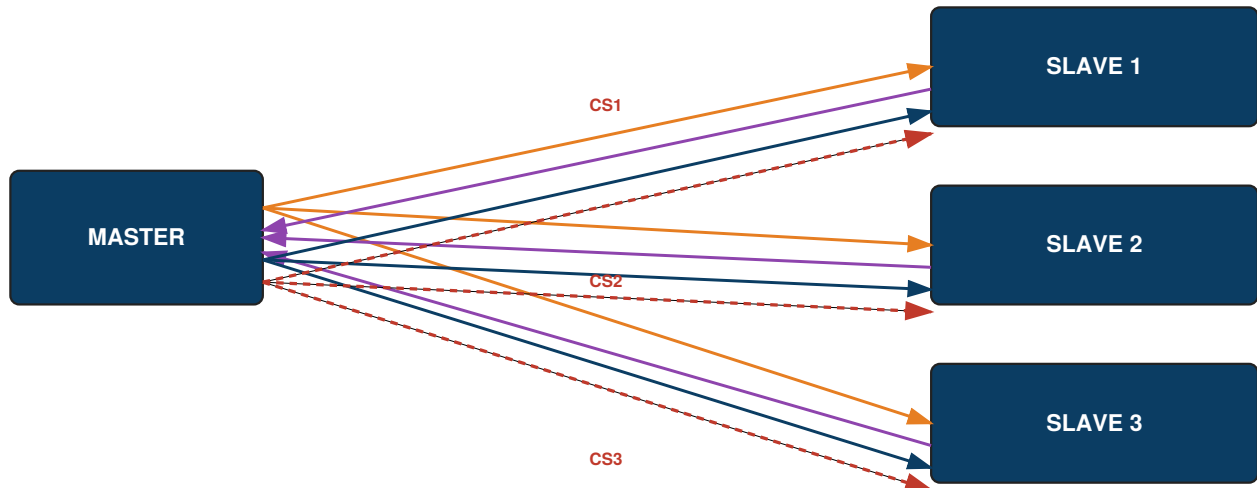
Two common conventions exist for multi-byte transfers, and datasheets vary — always check:

- **CS held low for the whole transaction:** CS drops once, N bytes are clocked through continuously, CS rises once at the end. This is the most common convention and is what most flash/EEPROM burst-read operations use.
- **CS toggled per byte:** some simpler/older devices expect CS to pulse high-then-low-again between every single byte, treating each byte as its own mini-transaction. Getting this wrong (holding CS low when the device expects per-byte toggling, or vice versa) is a very common bring-up bug when porting a driver between similar-looking parts.

Part 4 — Multi-Slave Topologies

4.1 Independent Chip-Select (most common)

SPI — Multi-Slave, Independent Chip-Select Topology



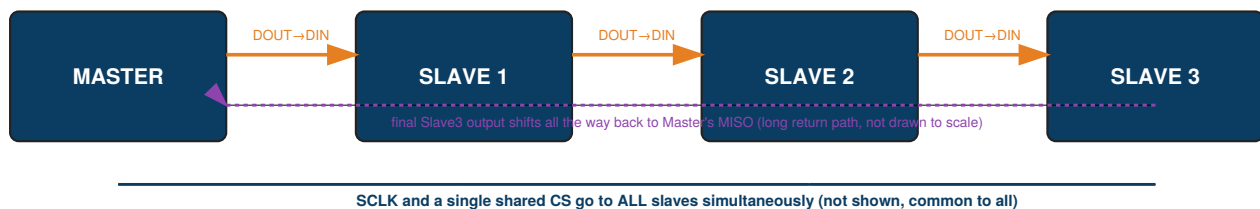
MOSI, MISO, SCLK are shared/bussed to all slaves. Each slave gets its OWN dedicated CS line — master asserts exactly one CS at a time.

MOSI, MISO, and SCLK are shared/bussed across all slaves (all slaves' MOSI pins are tied together, all SCLK pins tied together, and — critically — all slaves' MISO outputs are tied together onto one shared MISO line back to the master). Each slave gets its own dedicated CS line from the master. The master asserts exactly ONE CS line at a time; every other slave must keep its MISO pin tri-stated (high-impedance) while its own CS is inactive, so it doesn't electrically fight the currently-selected slave's MISO output.

This is why an SPI slave's datasheet timing diagram for CS matters so much: if a slave's MISO output doesn't tri-state fast enough after its CS goes inactive (or, worse, doesn't tri-state at all due to a hardware fault), it will corrupt every other slave's response on that shared MISO line — a classic hard-to-diagnose multi-device SPI bus bug.

4.2 Daisy-Chain Topology

SPI — Daisy-Chain Topology (Shift-Register Style)



Some SPI devices (common in LED drivers, shift-register I/O expanders, and some sensor chains) support daisy-chaining: the master's MOSI goes only to the first slave; that slave's data-output pin feeds directly into the next slave's data-input pin, and so on down the chain; only the LAST slave's output finally returns to the master's MISO. SCLK and a single shared CS go to every slave simultaneously.

To read/write N devices in a daisy chain, the master must clock out $(N \times \text{bits-per-device})$ total, since data effectively has to "ripple" all the way through every device in the chain before the response reaches the master. This trades wiring simplicity (no per-device CS lines needed) for higher latency and the requirement that all devices in the chain be clocked together as one long virtual shift register.

Part 5 — Practical Engineering Notes

5.1 Clock Speed Considerations

SPI has no protocol-mandated maximum speed — it's purely limited by the slowest device on the bus and the physical layer (trace length, capacitance, signal integrity). Datasheets specify a maximum SCLK frequency (e.g., "up to 20 MHz at 3.3V"); exceeding it risks the slave not latching data correctly even though the master appears to transmit fine, since there's no acknowledgment/CRC layer to catch such errors — unlike CAN, a mis-clocked SPI transaction can fail completely silently.

5.2 Common Bring-Up Issues

Symptom	Likely Cause
Reads always return 0x00 or 0xFF	CS not actually toggling (stuck high = slave never selected, or stuck low = slave thinks it's always selected and gets confused), or MISO not wired/floating
Data looks scrambled/shifted by one bit	Wrong CPOL/CPHA mode — classic symptom is data that looks "off by one bit position" because sampling happens on the wrong edge
Byte values look bit-reversed (e.g., expecting 0x0F, getting 0xF0)	MSB-first/LSB-first mismatch between master config and slave expectation
Works with one slave alone but fails when multiple slaves share the bus	A non-selected slave's MISO isn't properly tri-stating, corrupting the shared MISO line — check each slave's CS timing and tri-state behavior
Intermittent corruption only at higher clock speeds	Signal integrity issue — trace length/capacitance too high for the chosen SCLK frequency, missing series termination resistors, or ground bounce
First byte of every transaction is garbage but rest is fine	Slave needs a dummy/turnaround byte after command or address that the driver isn't sending (common with flash memory "Fast Read" style commands)

5.3 SPI vs I2C vs UART — Quick Comparison

Property	SPI	I2C	UART
Wires	4 (+1 per extra slave's CS)	2 (SDA, SCL) shared by all devices	2 (TX, RX), point-to-point only
Duplex	Full	Half	Full
Addressing	Physical (CS line) — no in-band address needed	In-band 7/10-bit address on the bus	None — inherently point-to-point
Clock	Synchronous, master-driven	Synchronous, master-driven (with clock stretching)	Asynchronous — both sides must agree on baud rate in advance
Typical speed	Several MHz to 10s of MHz	100 kHz-3.4 MHz (standard/fast/high-speed modes)	Up to ~1-few Mbit/s typical
Error checking	None built-in (application must add its own)	ACK bit per byte	Optional parity bit

Part 6 — Interview Questions & Answers

6.1 SPI Fundamentals

Q1. What are the four SPI signal lines and what does each do?

MOSI (Master Out Slave In — master-to-slave data), MISO (Master In Slave Out — slave-to-master data), SCLK (clock, always generated by the master), and CS/SS (chip select, tells one specific slave to participate).

Q2. Is SPI full-duplex or half-duplex? Explain what that means in practice.

Full-duplex — on every clock pulse, a bit shifts out of the master (MOSI) and a bit shifts out of the slave (MISO) simultaneously. In practice this means every "read" transaction still requires the master to clock out dummy bytes on MOSI, and every "write" transaction still produces (usually ignored) bytes coming back on MISO.

Q3. Why doesn't SPI need a shared ground reference to be discussed the way CAN's differential signaling does?

SPI is single-ended (each signal referenced to ground, not a differential pair like CAN_H/CAN_L), so a solid common ground between master and slave is still required for signal integrity, but there's no "differential" concept — each line's logic level is just measured against local ground directly.

Q4. Why is SPI generally faster than I2C?

SPI has no arbitration, no in-band addressing overhead, no per-byte ACK bit, and is a dedicated point-to-point (or CS-selected) link rather than a shared open-drain bus — all of this removes overhead that I2C's shared-bus, addressed, ACK'd protocol requires, letting SPI push much higher clock rates.

Q5. Does SPI have any built-in error detection like CAN's CRC?

No — SPI itself has zero error detection or correction. If you need data integrity checking, the application/device protocol layered on top of SPI must implement its own checksum or CRC (many flash memory and sensor devices do include an optional CRC byte in their command protocol, but that's device-specific, not part of SPI itself).

Q6. Why must CS go low before SCLK starts toggling?

CS falling is what tells the slave "a transaction is starting for me" — it's the trigger that un-tri-states the slave's MISO output and resets its internal bit/byte counter to the start of a fresh command. If clock pulses happened before CS was asserted, the slave wouldn't be listening yet and those pulses would be ignored (or worse, if CS was left asserted from a previous transaction, misinterpreted as a continuation).

6.2 Clock Modes (CPOL/CPHA)

Q7. What do CPOL and CPHA each control?

CPOL (Clock Polarity) sets the clock's idle level: 0 = idles low, 1 = idles high. CPHA (Clock Phase) sets which of the two clock edges per bit is the sampling edge: 0 = first edge, 1 = second edge.

Q8. What happens if the master and slave are configured for different SPI modes?

Data gets sampled at the wrong instant relative to when it's actually valid on the bus, corrupting every bit — this typically looks like scrambled, shifted, or seemingly-random data even though the wiring and clock frequency are both correct, and is one of the most common SPI bring-up bugs.

Q9. In Mode 0, on which edge is data sampled, and what's the idle clock state?

Clock idles LOW (CPOL=0); data is sampled on the rising edge (the first edge after CS goes active, since CPHA=0).

Q10. Why does data need to change on the edge OPPOSITE the sampling edge?

To give the signal time to propagate and settle (accounting for transceiver/trace delay) before it gets latched at the sample edge. If data changed on the same edge it's sampled on, there'd be a race condition where the receiver might catch the old value, the new value, or an invalid in-between transition depending on tiny timing variations.

Q11. A device's datasheet says "clock idles high, data valid on the leading (falling) edge." Which SPI mode is this?

Mode 2 (CPOL=1, CPHA=0) — idle high, and CPHA=0 means sampling happens on the first edge after CS asserts, which — since idle is high — must be a falling edge.

6.3 Transaction Structure & Practical

Q12. Does the SPI protocol itself define a frame format with fields like address, length, or CRC?

No — SPI only standardizes the electrical/timing behavior (CS, SCLK modes, bit order). Any command/address/data structure on top of raw bit-shifting is entirely defined by the specific slave device's datasheet, not by SPI itself.

Q13. Why might a flash memory device need a "dummy byte" between the address and the actual data in a read command?

The flash's internal memory array needs some processing/access latency (a few clock cycles) after decoding the address before the requested data is ready to shift out; the dummy byte's clock pulses are what "buy time" for that internal read to complete without the master having to guess a variable delay.

Q14. In a multi-slave bus with independent CS lines, why must non-selected slaves tri-state their MISO output?

Because all slaves' MISO pins are wired together onto one shared line back to the master. If a non-selected slave kept actively driving MISO, it would electrically contend with the currently-selected slave's output, corrupting the data (and potentially damaging output drivers if one drives high while another drives low).

Q15. What's the tradeoff of daisy-chain SPI topology versus independent chip-select?

Daisy-chain needs fewer master GPIO pins (no per-device CS), but every read/write must clock data through the entire chain (higher latency, and all devices must be touched together), whereas independent CS lets the master address any single device quickly and independently, at the cost of one extra GPIO per slave.

Q16. You're reading a sensor register over SPI and getting a plausible-but-wrong value every time. The wiring, clock mode, and clock speed all check out. What else would you check?

Bit order (MSB-first vs LSB-first mismatch — this produces a plausible-looking but bit-reversed byte, not obvious garbage), whether the correct number of dummy/turnaround bytes are being sent per that device's specific command protocol, and whether the register address itself (and any read/write flag bit within it) is being formed correctly per the datasheet.

Q17. Why is it important to check a slave's maximum SCLK frequency, given SPI itself has no error-checking to flag when you exceed it?

Because exceeding the slave's rated maximum clock speed can cause silent data corruption — the slave may not be able to latch/shift data reliably at that speed, and since SPI has no CRC/ACK to catch this, the master may simply receive incorrect data with no indication anything went wrong, making it a subtle root cause to trace back to during debugging.

Q18. How would you debug an SPI bus where a newly-added second slave causes the first (previously working) slave to intermittently read incorrect data?

Suspect a shared-MISO contention issue: verify the new slave's CS is being correctly toggled by the master and that it's not accidentally tied active (stuck-select), confirm on a scope/logic analyzer that the new slave's MISO actually goes high-impedance when its CS is inactive, and check that both slaves are configured for compatible SPI modes if any master-side mode-switching per-device is expected.

Q19. What is the practical difference between SPI's CS-based addressing and CAN's ID-based addressing?

SPI addressing is purely physical — a dedicated wire per device, decided by the master's GPIO control, with zero in-band protocol overhead. CAN addressing (via message ID) is logical/in-band, broadcast to every node on a shared bus, and requires each node to filter messages by content rather than being electrically selected — this is also why SPI scales poorly to many devices (one pin each) while CAN scales to many nodes on the same two wires.

Q20. Why can't multiple SPI masters share the same bus the way multiple CAN nodes can all be masters?

SPI has no arbitration mechanism — SCLK and CS are actively driven single-ended outputs from one specific master. If two masters tried to drive SCLK or CS simultaneously, they would electrically contend and corrupt the bus, since there's no bitwise-arbitration or collision-detection scheme like CAN's to resolve it. SPI is fundamentally designed around exactly one bus master at a time (multi-master SPI variants exist but require external arbitration logic that isn't part of the standard).